EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 5-IDA: IDA Pro — Decode, Hack, and Patch IEEE 754 Floats and Doubles (Raw `.bin`)

ARM Cortex-M33 • GDB • Ghidra

October 20, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the two lesson projects with `Release` and get both an `.elf` and a raw `.bin`
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load the raw `.bin` into IDA at `0x10000000`
- Read how a `float` / `double` constant is materialized from the compiler's literal pool these two programs
- Reconstruct a 64-bit `double` from the ABI register pair `r2:r3` and hack it live
- **Resolve the functions in the IDA GUI** using the ELF symbol map, including the `pico_double` formatting helpers `printf` pulls in
- **Patch** the constant bytes, export the image, convert to UF2, and export it
- Prove why `42.5 -> 99.0` is a one-word patch but `42.52525 -> 99.99` needs two

How This Guide Works

The build produces two files for each project:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table	Ground truth for every function address and name
`.bin`	The raw flash image, no headers, no symbols	The image we load into IDA and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in IDA, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the Week 5 lesson, and the Week 5 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it folds the `float` / `double` initializer into the literal pool and links the `pico_double` formatting helpers straight into the `printf` path. If you build `Debug`, the SDK function addresses move and the float/double helpers are laid out differently, so nothing lines up. Always build `Release` for this lesson.

The workflow is static-only: import the raw image, recover the code, decompile it, and patch the artifact:

d prove what the code does. ge. 3. Resolve the functions in IDA using the ELF symbol map. 4. Patch the bytes, export, convert and export.

Project	Prints	The hack
`0x000e_floating-point-data-type`	`fav_num: 42.500000`	change the printed double `42.5` to `99.0` (one word)
`0x0011_double-floating-point-data-type`	`fav_num: 42.525250`	change the printed double `42.52525` to `99.99` (two words)

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

> `main` **is not at the same address in both projects this week.** It is `0x10000234` in Project 1 and `0x10000238` in Project 2, because each program materializes its constant slightly differently. We still anchor to `main` through the one byte-identical place that always names it: the middle `blx` in `platform_entry` (Step 12).

Part 1: Build and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **IDA Pro** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

Glossary

Term	Definition
`.bss`	Section for uninitialized global variables; zeroed by startup code

Term	Definition
`.data`	Section for initialized global variables; copied from flash to SRAM at boot
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
`.rodata`	Read-only section for constants and string literals; stays in flash
Bias	Constant added to an IEEE 754 exponent (`127` for float, `1023` for double)
Double	64-bit IEEE 754 floating-point type (1 sign, 11 exponent, 52 mantissa)
Float	32-bit IEEE 754 floating-point type (1 sign, 8 exponent, 23 mantissa)
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
ly flash	
IEEE 754	The standard that defines binary floating-point encoding
Inlining	The optimizer replacing a function call with the function body
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr` / `ldrd`
Mantissa	The fractional significand bits (23 for float, 52 for double)
MMIO	Memory-mapped I/O — hardware registers accessed as memory addresses
Promotion	C's automatic `float` -> `double` conversion for variadic arguments
Register pair	Two 32-bit registers (`r2:r3`) that together hold a 64-bit value
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at `0xd0000000`
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Verify the static behavior from the recovered instructions.5`, two words for a repeating one like` 42.52525` — and export the patched artifact.